Software Task Round

Autonomous Ground Vehicle

For the selections, **six (6)** tasks are given. For each task, you first need to understand the problem statement, read concepts from the internet (some resources are provided in this doc itself), and then implement (code) it.

The task explanations are given at the end of this document in a separate section.

1. Make your programs Object Oriented if possible, although this is not a requirement.
2. You are expected to maintain proper documentation in notion for each task even if you are not able to complete them.
3. The resources given are not sufficient and you are expected to find more on your own.
4. You may discuss among yourselves and help each other, but sharing your code is a strict no. The use of plagiarized code will result in complete and immediate disqualification for both candidates, from whom the code has been copied and who have copied it.
5. For the tasks that require it, you can visualize the path or data using Matplotlib or OpenCV.
6. Participants may utilize any resources – including AI tools (LLMs), books, or videos – provided they fully document all sources and demonstrate understanding of their application.
7. Don't hesitate to contact any of us in case of any obstacles.
8. We have allotted the time with sufficient consideration for various factors and hence cannot give any extension to anyone as it would be extremely unfair to the rest of the candidates

A word of advice- You are expected to do at least one task, but you are encouraged to do more - this will increase your chances of getting selected and will also help you explore your interests in different fields we work on and get a general idea.

Mandatory Docker Setup

This is compulsory for everyone. Read up on basic docker documentation - It will help you in copying local files to the container and do other basic maneuvers. You must do all tasks in the docker container and push them finally for evaluation.

Windows Setup:

- Docker Installation: Install Docker Windows
- Install VcXsrv: VcXsrv is an X server that enables GUI applications from a Docker container to be displayed on Windows. Launch XLaunch and configure it with the default suggested options to start the X server
- Run the Docker Container with GUI Access:

  ```
  docker run --name agvdocker -e DISPLAY=host.docker.internal:0 -it agviitkgp/task:24
  ```

macOS Setup:

- Docker Installation: Install Docker macOS
- Install XQuartz: Restart your system after installing XQuartz, (reference)
- Run the Docker Container with GUI Access:

  ```
  docker run --name agvdocker -e DISPLAY=$(ipconfig getifaddr en0):0 -it agviitkgp/task:24
  ```

Running Docker:

- After the first run, use this command to restart and attach to the container:

  ```
  docker start -ai agvdocker
  ```

- Exit the Docker Safely:

  ```
  exit
  ```

- VSCode Setup: Refer this link

1. Aw Shii, Here we flow again...

Seeing Motion Like Humans Do

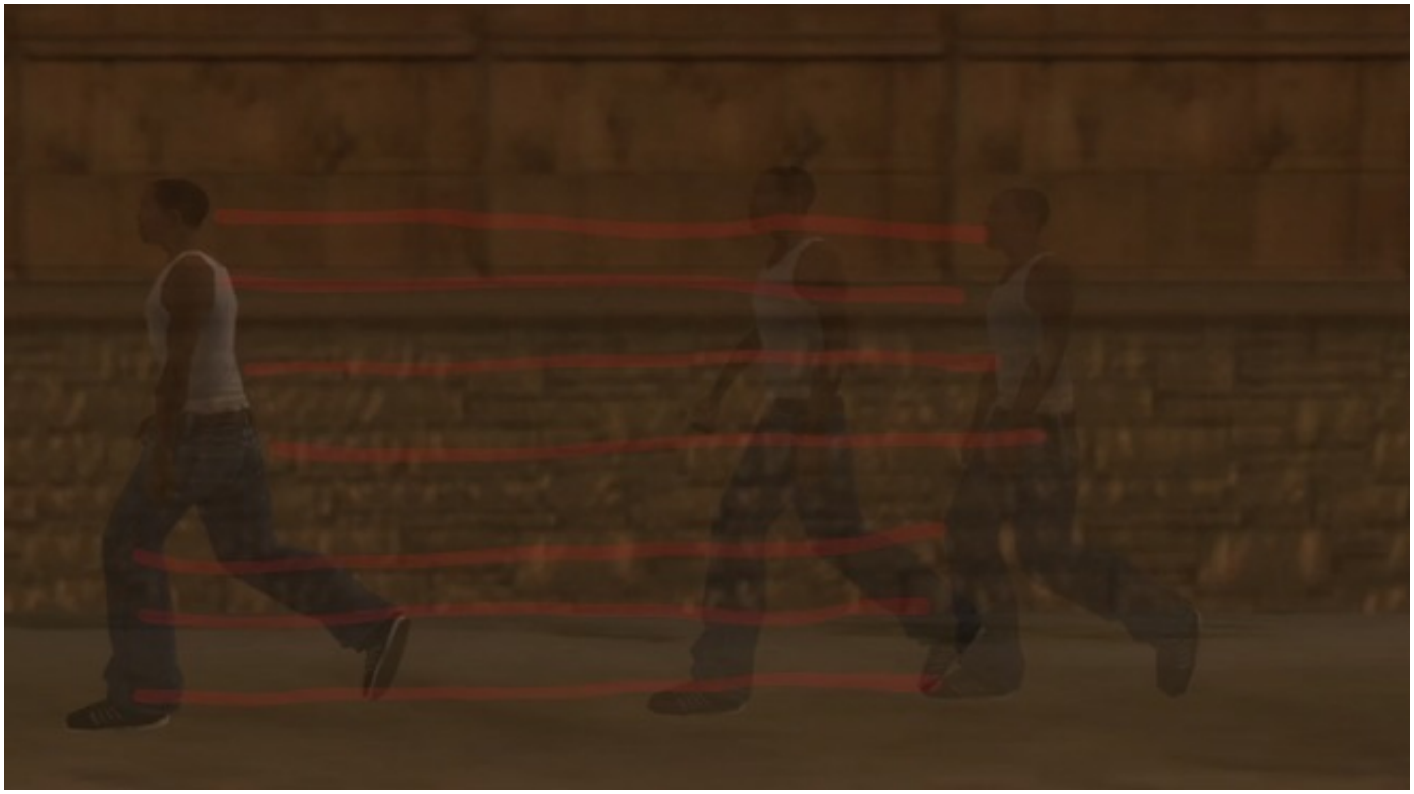
Think of merging onto a highway at dusk: your brain instinctively tracks the speed of approaching cars using subtle visual cues, not raw speedometers. Optical flow replicates this biological motion sense for machines, solving critical problems other sensors miss.

- Sudden Obstacles: Like spotting a skateboarder cutting across traffic (as in Chalmers University's collision detection system), optical flow detects approaching objects 30% faster than frame-by-frame detection.
- Parking Lot Navigation: Ever parallel park by gauging car bumpers' movement in your mirrors? Flow vectors create real-time "distance maps" between vehicles.
- Rainy Night Driving: New methods like SemARFlow maintain 85% accuracy in downpours where LiDAR fails—critical when windshield wipers struggle.
- Key Insight: Humans use motion cues first, details later—optical flow brings this biological advantage to machine.

SUBTASK 1: IMPLEMENTING OPTICAL FLOW

CJ NEEDS HELP!

You are expected to implement the Lucas-Kanade sparse optical flow estimation algorithm specified in this article on clips of your choice from this video. You are allowed to use basic OpenCV functions: *cv2.imread*, *cv2.imshow*, etc., and you may additionally use *cv2.goodFeaturesToTrack*. You may refer OpenCV documentation.



BONUS: Implement dense Optical Flow estimation algorithm on the video.

SUBTASK 2: GOING WITH THE FLOW

Let us now dive into a very interesting application of optical flow methods: moving your ego (a robot or an autonomous car) towards a target while avoiding obstacles. Your task here is to use the Lucas-Kanade algorithm in conjunction with a few methods in order to implement a make-shift controller that reaches a given goal in a cool simulator.

Before proceeding with the task, you are required to pull and set up the docker container as mentioned in the compulsory task. Operate only from the home directory.

Launch the container and do a pip install of pybullet ,then create a python file where you will be coding up most or all of the following task.

Objective: Use sparse optical flow (Lucas-Kanade method) to navigate to the end of the racetrack avoiding the obstacles in the provided setup (using the given script).

You must follow these steps (refer to the paper linked below for more clarity):

- Set up a camera feed and capture RGB frames from Pybullet's agent camera.
- Compute optical flow between consecutive frames to identify motion vectors.
- Detect Focus of Expansion (FOE) to infer agent motion direction and distinguish ego-motion from obstacle motion.
- Generate a visual potential field where obstacles create repulsive vectors and the target (e.g., a doorway) creates an attractive vector.
- Integrate with Pybullet's API to steer the agent using gradient based control.

  You must go through this this paper and understand its math (at least at a high-level) to accomplish this. Apart from your implementation of Lucas-Kanade method, you may use any library function from OpenCV and NumPy. Go through their documentation.

  Use as many resources as you want *(including LLMs)* for this subtask.

Figure 1: Camera feed

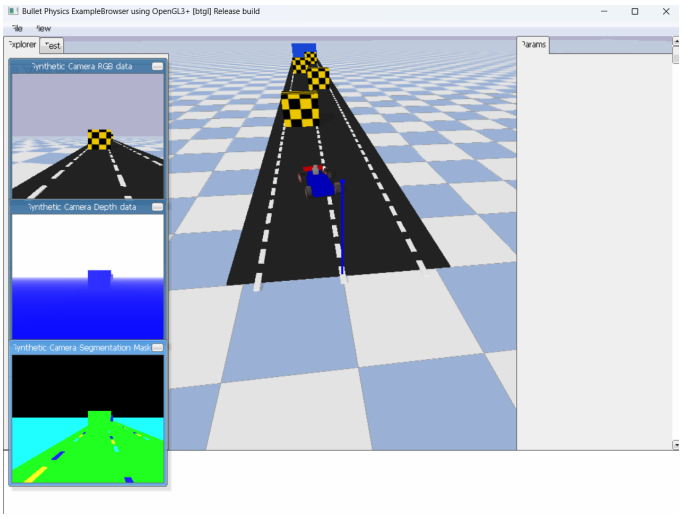


Figure 2: Setup

2. I Won't Hit On You !

Introduction

In the realm of autonomous vehicles, the ability to navigate safely through dynamic environments is crucial for reliable and efficient operation. One key component of this capability is dynamic obstacle avoidance, which involves detecting, tracking, and maneuvering around moving objects in real-time. This process plays a vital role in enabling autonomous vehicles to make split-second decisions, prevent collisions, and ensure the safety of both passengers and other road users.

Running the Simulator

You are given a **simulator** in C++ that abstracts away most physical and simulation details and provides an API where you can directly program an agent's planner and controller. You can follow the instructions in the README of the repository to install and set the simulator up.

The tasks in this simulation involve implementing and fine-tuning controllers for an autonomous system navigating through a simulated environment. The objective is to develop a controller capable of handling static waypoints, dynamic obstacles, and noisy sensor data. Below are the three tasks:

Task 1: Basic Static Controller Implementation

- Using the calculate_1 function, obtain the world map, triangulate the free area and treat the centroids of the triangles obtained as waypoints, now implement a basic static controller, such as Follow the Gap, PID, or Pure Pursuit, to follow the waypoints you found in an arbitrary order.

Task 2: Dynamic Obstacle Avoidance Controller

- Modify the movementspecifier array of your simulation instance and assign arbitrary movements to every obstacle, you can find examples on how to do this in the README, now modify your controller to account for dynamic obstacles and navigate around them.
- The simulation environment will contain both dynamic and static obstacles. Select an appropriate algorithm to optimize performance and robustness against edge cases. Be as creative with this as you can.

Task 3: Noisy LiDAR-Based Dynamic Obstacle Avoidance

- You can add noise to your LiDAR scans by setting the default parameter in the agentraycast method in the simulation class, set it to a small positive value and modify the dynamic obstacle avoidance controller to function effectively in an environment with **noisy LiDAR data**.

3. Lost & Found: Multi-Agent Rendezvous

When Worlds Collide (Literally)

- Multi sensor SLAM techniques are used to merge 3D maps generated by UAVs and UGVs to generate detailed disaster environment maps that aid search and resuce operations .
- ETH Zurich's uses swarms of aerial robots to achieve a 92% accuracy in shared map alignment for collaborative construction.
- In nature ants use pheromone trails to create shared pathways and wolf packs use triangulation while hunting down prey to compile information gathered from multiple sources.

Task: Build An Autonomous Multi-Agent Navigation Pipline

You are given a **simulator** in C++ that abstracts away most physical and simulation details and provides an API where you can directly program an agent's planner and controller based on sensor and mapping data, for each successive sub-task you will have access to less and/or lower quality data than the previous one and will need to employ progressively more sophisticated techniques.

The task is designed broadly for you to understand the concepts behind **localization, map-merging and autonomous navigation and planning** in GPS-denied environments. The robots are equipped with LiDAR sensors and you have complete control over the instantaneous inputs to the throttle and steer of your agent at any point.

Templates for the planning mapping and control code you're supposed to write are given in the *main.cpp* file and you can go through *simulation.hpp* to understand how they are used . Your task is to implement a planner and a mapping algorithm for each agent and devise a method to merge the their into one to construct a full map which you must store and display.

For your convenience, the task is split into 5 related and progressively more difficult sub-tasks, solving any sub-task fully will give you credit for all previous sub-tasks.

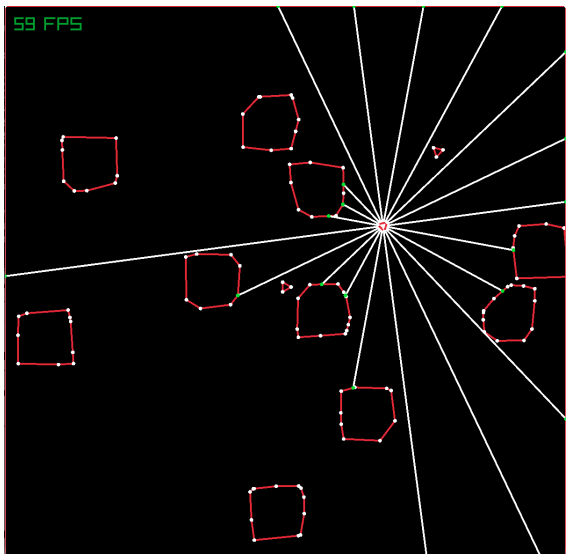
1. Write a simple controller for the agent, i.e. implement a function that you use inside the agent's calculate function to compute the steer and acceleration based on where you want to move, you may find it worthwhile to look into PID control, now use this controller to move around in the map with the calculate_1 function, which gives you access to the entire map and the locations and velocities of all other agents by default.

2. Use the controller you developed in the previous part and augment it with an exploration algorithm like PRM or RRT to map the entire map area with one agent while using the calculate_2 function for control this time, this only gives you access to the state of the current agent and the LiDAR scans, once you're done mapping the area display the map you computed in a separate window.

3. Extend your algorithm from the previous subtask to two agents and combine the maps with an algorithm of your choice. Display it similarly to the last part, since you are using more agents to explore this time, try to cut down on the time you need to explore the whole map.

4. Change the error standard deviation parameter in the agentraycast function in the simulation class to a (small)non-zero value, this makes your LiDAR readings noisy, redo the previous two sub-tasks with this noisy data.

5. The simulator is capable of handling an arbitrary number of agents, try extending your algorithm from sub-task 3 to 3 or more agents and using them to decrease the time you need to explore the entire map.



Deliverables (What we evaluate for):

- *How well-organized your pipeline is.* A major problem in non-robust solutions arise when one part of the pipeline or code fails - either due to a bug in the code itself or due to extrinsic factors - like compute limitations or other real-life deployment failures. Your code must be built to handle such scenarios where one failure doesn't impact the other parts.
- *Understanding and Originality.* Your solution must be well-thought-out and researched. And by virtue of the same, you must have complete understanding of your pipeline as well as implementation details of your code.
- *Generalizability and Corner Case Testing.* We will stress test your solution. So, lazily put-together solutions built solely through clever (or even worse un-clever) prompt engineering may not pass these cases. On other occasions, stress testing may also reveal bugs in say, code that you have written by yourself. The real reason for this deliverable is thus not only for weeding out lazy solutions, but also to test if you have accounted for corner cases that may very well happen during real-life deployment.
- *Novelty.* Introducing very rarely used algorithms that turn out to be efficient or coding up some new changes to existing popular algorithms will surprise us. So, be creative and innovative!

4. VizDoom

Introduction

While the task that follows below does not directly relate to autonomous vehicles, a lot of the ideas and concepts that you will use to solve it will directly carry over to autonomous vehicles and automation in general. This includes but is not limited to planning in continuous spaces, designing controllers to carry out your planned trajectory, and using simulators and integration so that you can test out your algorithms before deploying them in the real world.

For this task, you will be using ViZDoom, which is an AI research platform/simulator based on the game which arguably birthed the FPS genre - Doom. You can visit their GitHub repo here. Follow the instructions there to set it up on your machine (which is just a simple pip install for the most part) and go through the documentation and examples on how to use it. The overall goal of this task is to navigate a maze and reach a checkpoint.

Level 1

Load this custom .wad file into the simulator. A WAD file is a game data file used by Doom and Doom II, as well as other first-person shooter games that use the original Doom engine. On correctly loading the .wad file, you will see something like this:

Global Planning

You can either use the automap from the simulator or use the map directly from here. The white pixel is the initial position of your ego and blue is where you will find the blue skull needed to finish the level. You can use any planning algorithm you like to plan a trajectory but we prefer RRT*. You will need to keep an eye out for dynamic and kinetic constraints of your ego since any path connecting the two points might not be feasible for your controller to follow in the next level.

Trajectory Following

You will need to now translate your global trajectory to actions in space and direct your ego to the blue skull. You might need a closed loop controller that corrects the errors that can build up in following the trajectory.

Level 2

You will no longer have access to the automap or a predefined map. You are only allowed to use the data from the buffers. The objective remains the same. You will most likely need to implement some sort of DFS search in the space while keeping approximate track of where you are, controls for backtracking and behaviors to search for open passages; however, you are free to take any approach you like. An example depth buffer is given below:

Submission

To make your submission for this task, use this modified version of the ViZDoom repository. The examples folder contains two python files, for Level 1 and Level 2 of this task. You can add your code in these files and submit them. We have already set up the .wad file of the custom map in this repository and made changes in the configuration files accordingly.

Feel free to run and change the existing scripts in the examples folder to get a better understanding of the environment.

In addition to the mentioned codes, submit an image of the final global path that your designed algorithm suggested for the given map image. Submit a screen recording of the next parts of the task.

5. Locate Me If You Can

When Millimeters Matter

Imagine an autonomous hospital delivery robot navigating ICU corridors - one flawed sensor reading could mean collision with critical equipment. This challenge captures the essence of robust localization: transforming noisy LiDAR data into millimeter-precision positioning without GPS.

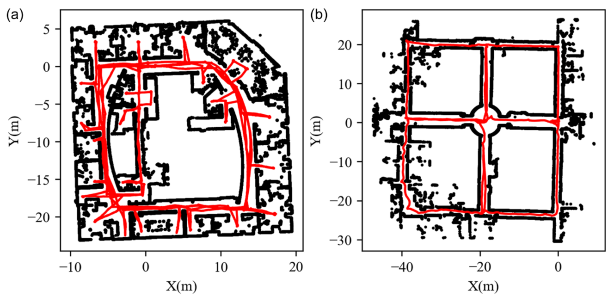
Task: Normal Distribution Transform for Localization

The task involves reading and implementing the research paper titled The normal distributions transform: a new approach to laser scan matching by P. Biber, W. Strasser (link to the research paper). You will have to implement the localization method using the Normal Distributions Transform (NDT) technique on this dataset.

SUBTASK 1: Use the dataset above to localize the agent on this map (use OpenCV to replace green pixels with white ones). Here are basic instructions on how to read the data (Or, you could try to parse the data yourself) [You might also find it useful to first understand the relation between the coordinates of the map and the dataset].

SUBTASK 2: Generate the NDT Density Map utilizing the formulas mentioned and simulate the movement of the agent at each step, optimizing the pose using Newton's Optimization. [Note that the Gradients may need to be modified a bit.]

NOTE: Read the Research Paper thoroughly for the interview even if you can't completely implement the paper. You may replace problematic parts of the implementation with simpler alternatives.



6. Reconstruction and Odometry

SUBTASK 1: SPARSE 3D RECONSTRUCTION

Part 1: Introduction

In the realm of autonomous ground vehicles, the ability to perceive and understand the surrounding environment accurately is crucial for safe and efficient operation. One key component of perception is 3D reconstruction, which involves creating a detailed and comprehensive representation of the scene in three dimensions. This process plays a vital role in enabling autonomous vehicles to make informed decisions, accurately detect objects, navigate complex environments, and effectively plan their trajectories.

Task

This task requires you to construct a 3D representation of surroundings from multiple view images. A detailed description of the concept behind the task and how you are expected to tackle it are given in the Link . All the required data is also given in the above link.

3D Reconstruction

SUBTASK 2: Monocular Visual Odometry (3D-to-2D Pipeline)

The core of this reconstruction task lies in estimating the camera's trajectory through space. While initial frames may require triangulation, the continuous estimation of the camera pose is achieved through a 3D-to-2D tracking pipeline. This task is more of a practical implementation. Given only the raw dataset, you are responsible for the entire pipeline from feature correspondence to 3D mapping mirroring a real SLAM setup.

Learning Resources

To build a solid theoretical foundation, refer to the following resources:

- **Basic VO Theory:** Visual Odometry Part I: The Basics (Scaramuzza) - Explains the geometric pipeline and feature-based motion estimation.
- **Advanced VO Theory:** Visual Odometry Part II: The Basics (Scaramuzza) - Contains more details about parts of the pipeline.

3.1 Feature Tracking and Correspondence

To determine how the camera has moved between frames, we must track consistent landmarks in the environment. Use the Subtask1 code here.

- **Detection:** Use the **Shi-Tomasi** corner detector to identify high-quality trackable features.
- **Tracking:** Implement the **Lucas-Kanade Optical Flow** algorithm to find correspondences between the previous 2D image coordinates and the current frame.

3.2 Perspective-n-Point (PnP)

The primary objective is to estimate the current camera pose given a set of known 3D points in the world and their corresponding 2D projections in the image plane.

- **The PnP Problem:** You are tasked with finding the Projection Matrix P. By

minimizing the reprojection error (the distance between where a 3D point should appear and where it actually appears in the image), you solve for the camera's extrinsic parameters.

- **Decomposition:** Once the projection matrix P is found via a least-squares solution, decompose it using the provided intrinsic matrix K to extract the Rotation (R) and Translation (t).

3.3 Robustness in Dynamic Environments

Standard least-squares solvers fail when the scene contains dynamic obstacles (moving cars, pedestrians). To maintain a stable ego-motion estimate:

- **RANSAC (Random Sample Consensus):** Integrate RANSAC into your PnP solver. This will iteratively select subsets of points to find the "best fit" pose that represents the stationary background, effectively filtering out "outlier" points from moving objects.

3.5 Testing and Verification

You can test your implementation on the video file given here TUM RGBD dataset. You are provided with:

- Standardized camera intrinsic parameters (K).
  fx = 517.3
  fy = 516.5
  cx = 318.6
  cy = 255.3

You do not need to evaluate the performance, but you must visualize the trajectory using the odometry_visualizer.py given
Note: It is recommended not to test it on all frames everytime as loading and processing may take too much time. you can test it on approx 50 frames or any comfortable number initially

Contact Information

Name	Phone Number	Email
Akshat Shukla	+91 87699 27445	shuklaakshat.217@gmail.com
Uday Kalyan S	+91 8197205429	udaykalyansreenivasa@gmail.com
Kaustubh Kaka	+91 70113 02106	kaustubhkaka@gmail.com
Tanishq Saxena	+91 8766227047	tanishqsaxena.in@gmail.com
Oishi Nandi	+91 8141204296	oishi.nandi22@gmail.com
Netra Nandankar	+91 8433810625	nandankarnetra5@gmail.com